Jogo da velha em C

Estruturas de Dados e Alocação Dinâmica

Thiago Rafael Kraus

Minicurso de Programação em C

29 de março de 2026

Arquitetura e Responsabilidades do Módulo Lógico

- O backend implementado em C atua como o **módulo central** do sistema, detendo as regras de negócio.
- **Desacoplamento de Interface:** O código opera de forma agnóstica em relação à camada de apresentação (*View*).
- **Atribuições do Subsistema:**
  - Gerenciamento do estado do sistema em tempo de execução.
  - Validação de integridade das entradas (movimentos).
  - Processamento de algoritmos de verificação de condição de parada (vitória/empate).
  - Provisionamento e liberação de recursos na memória estática e dinâmica (*Heap*).

1. Modelagem do Estado do Sistema

Substituímos literais em código (*hardcoded*) por tipos enumerados para garantir segurança de tipo e clareza semântica. O estado global é encapsulado em uma estrutura unificada.

```c
// Tipo enumerado para defini  o de dom nio de valores
typedef enum {
    VAZIO = 0,
    JOGADOR_X = 1,
    JOGADOR_O = 2
} ValorCasa;

// Estrutura de dados para o estado da aplica  o
typedef struct {
    ValorCasa *tabuleiro; // Ponteiro base do segmento alocado
    int jogadas_realizadas; // Vari vel de controle de itera  es
} EstadoJogo;

static EstadoJogo *jogo_atual = NULL; // Encapsulamento em escopo de m dulo
```

2. Alocação Dinâmica na Memória Heap

Utilizamos a diretiva `malloc` para a requisição de blocos contíguos de memória durante o tempo de execução (*runtime*).

```
void iniciar_jogo() {
    // 1. Provisionamento de mem ria para a estrutura de controle
    jogo_atual = (EstadoJogo*) malloc(sizeof(EstadoJogo));

    // 2. A loca  o do vetor de estados (9 posi  es de tamanho enum)
    jogo_atual->tabuleiro = (ValorCasa*) malloc(9 * sizeof(ValorCasa));
    jogo_atual->jogadas_realizadas = 0;

    // 3. Inicializa  o do bloco de mem ria via itera  o de ponteiros
    for (int i = 0; i < 9; i++) {
        *(jogo_atual->tabuleiro + i) = VAZIO;
    }
}
```

3. Transformação Linear de Coordenadas (2D $\rightarrow$ 1D)

A matriz de 3×3 é representada fisicamente na memória como um vetor contíguo de 9 elementos. O acesso a um elemento requer uma transformação linear do espaço bidimensional (l, c) para um índice unidimensional.

A função de mapeamento de índice:

$$I(l, c) = l \cdot N_c + c$$

Onde N_c é o número total de colunas (3).

Exemplos de resolução vetorial:

- Elemento na Linha 0, Coluna 2 $\rightarrow I(0, 2) = (0 \cdot 3) + 2 = 2$
- Elemento na Linha 1, Coluna 0 $\rightarrow I(1, 0) = (1 \cdot 3) + 0 = 3$
- Elemento na Linha 2, Coluna 2 $\rightarrow I(2, 2) = (2 \cdot 3) + 2 = 8$

4. Resolução de Endereços e Manipulação de Memória

A validação da entrada é feita resolvendo o endereço físico da célula alvo, utilizando aritmética de ponteiros e desreferenciação para leitura e escrita.

```c
int preenche_pos(int jogador, int linha, int coluna) {
    // Transforma  o linear das coordenadas de entrada
    int indice = (linha * 3) + coluna;

    // Resolu  o do endere o de mem ria da c lula espec fica
    ValorCasa *casa_alvo = &jogo_atual->tabuleiro[indice];

    // Desreferencia  o (*) para avaliar o conte do alocado
    if (*casa_alvo == VAZIO) {
        *casa_alvo = (ValorCasa)jogador; // Coer  o de tipo (Cast)
        jogo_atual->jogadas_realizadas++;
        return 0; // C digo de sa da: Sucesso
    }
    return -1; // C digo de sa da: Falha (C lula ocupada)
}
```

5. Desalocação e Prevenção de Memory Leaks

O ciclo de vida da memória dinâmica exige que toda região provisionada via `malloc` seja explicitamente liberada ao sistema operacional através da função `free`.

```
void finalizar_jogo() {
    if (jogo_atual != NULL) {
        // A libera  o deve ocorrer do bloco mais interno para o externo
        free(jogo_atual->tabuleiro); // 1. Desaloca o vetor cont guo
        free(jogo_atual);            // 2. Desaloca a estrutura de estado
        jogo_atual = NULL;           // 3. Mitiga  o de ponteiros pendentes (Dangling Pointers)
    }
}
```

```c
// fim?
if (aprendeu) continue;
else {
    try_again();
}
```